

NODE CALIBRATION

Smart Signage System

Department of Software Development, I2ST.

Abstract

The management of distributed hardware nodes in modern smart signage systems demands a unified, efficient, and secure configuration platform. This project presents a comprehensive Windows Desktop Application, developed in C# using the .NET framework, designed to centrally manage and calibrate ESP32-based controller units deployed across multiple signage panels. The application integrates a local SQL Server database for persistent storage of node records, user credentials, and panel location descriptions, combined with a JSON-based settings file for application-level preferences.

The system provides a role-based authentication mechanism, ensuring that only authorized personnel can access critical functions. A dynamic user interface allows operators to define network nodes with unique parameters including IP addresses, panel serial numbers, and physical locations. Two distinct communication channels—direct USB serial and Over-the-Air HTTP—enable the deployment of configuration payloads to remote devices without the need for physical access. The application enforces data integrity through strict validation rules, preventing duplicate IP assignments and ensuring each hardware unit is calibrated only once. An integrated serial monitor and OTA management panel offer real-time feedback on device status and configuration success.

The ESP32 firmware, implemented in the Arduino framework, incorporates Ethernet connectivity, an HTTP server, MQTT telemetry, and persistent storage via the Preferences library. On receipt of a configuration, the device dynamically updates its network settings, sensor calibration constants, and operational parameters, then automatically restarts to apply changes. A factory reset capability allows remote recovery of misconfigured devices.

This solution significantly reduces the time and complexity involved in provisioning and maintaining a large network of signage controllers. It eliminates manual data entry through planned Excel import/export functionality and lays the groundwork for future integration of Modbus TCP-based diagnostic testing. The project demonstrates a robust, scalable, and practical approach to industrial device management, blending desktop application engineering with embedded systems communication.

CONTENTS

Chapter No.	Title	Page No.
Chapter 1	Introduction.	
1.1	Background.	
1.2	Proposed System.	
1.3	Scope and Objectives.	
Chapter 2	Requirement specification.	
Chapter 3	System Design.	
3.1	System Architecture.	
3.2	Flow Diagrams.	
3.3	Database Schema.	
Chapter 4	System Implementation.	
4.1	Main Window Interface.	
4.1.1	Menu Strip (Top Navigation Bar).	
4.1.2	Left Navigation Panel.	
4.2	Authentication Module.	
4.3	Node Management Module.	
4.4	Area Management Module.	
4.5	Panel Settings Module.	
4.6	Settings Module.	
4.7	Network Configuration Module.	
4.8	I/O Activity Module.	
4.9	Upload/Download Module.	
4.10	Tests Module.	

Chapter 1

INTRODUCTION

1.1 BACKGROUND

Modern metro rail networks rely heavily on digital signage systems to convey real-time passenger information, safety instructions, and operational status across platforms and concourses. These signage systems are typically driven by embedded controllers installed at numerous physical locations, often numbering in the hundreds across a single line. Each controller must be individually configured with network parameters, calibrated to its specific sensors and power supply, and continuously monitored for faults. Traditionally, this provisioning and maintenance work has been carried out manually by field engineers who connect a laptop directly to each device via USB, a process that is time-consuming, error-prone, and difficult to audit.

As metro systems expand and demand higher reliability, there is a clear need for a centralized tool that can manage these edge nodes remotely, enforce consistent configuration, and integrate seamlessly with existing industrial control infrastructure. The project described in this document addresses that need by providing a Windows Desktop Application purpose-built for the calibration and management of smart signage controllers in an operational metro environment.

1.2 PROPOSED SYSTEM

The proposed system is a full-featured Windows Forms application that consolidates all node management functions into a single interface. It provides role-based authentication, ensuring that only authorised personnel can modify safety-critical parameters. The left panel of the main window contains navigation buttons for dedicated modules: Home, Node, Area, Tests, Settings, Network, I/O Activity, Upload/Download, and Panel Settings. Each module is implemented as a self-contained User Control that is dynamically loaded into the central content area, preserving a clean separation of concerns.

The application stores user credentials, node definitions, and panel location descriptions in a Microsoft SQL Server Local DB database. A JSON file retains technician contact details and maintenance preferences. Communication with the ESP32-based edge nodes occurs over two channels managed by the Panel Settings module: a USB serial interface for bench provisioning and an Over-the-Air HTTP endpoint for remote configuration. A consistent JSON payload carries the node's route, panel location, IP address, description, panel serial number, and optional calibration constants. If calibration fields are not supplied, they are omitted from the payload, preserving the existing device calibration. A factory-reset function, triggered by a dedicated JSON flag, clears all stored settings and returns the device to its default state.

By abstracting the hardware complexity behind a straightforward user interface, the system reduces commissioning time, eliminates transcription errors, and provides an auditable workflow suitable for twenty-four-hour, seven-day-a-week metro operations.

1.3 SCOPE AND OBJECTIVES

The scope of this project is to design, implement, and document a Windows Desktop Application that accomplishes the following objectives:

- Provide a secure, role-based authentication mechanism that distinguishes between administrators and standard operators.
- Enable full Create, Read, Update, and Delete operations on node records, where each node is uniquely identified by a panel serial number and assigned to a specific route and panel location.
- Allow field engineers to attach human-readable physical descriptions to each of the four hundred possible panel slots (four routes of one hundred panels each).
- Deploy configuration payloads to ESP32 controllers over USB serial or over-the-Air HTTP, with an integrated serial monitor for real-time feedback and a factory-reset capability.
- Enforce data integrity through validation rules that prevent duplicate IP addresses, duplicate panel serial numbers, and conflicting panel location assignments within a route.
- Restrict the deployment of sensor calibration values to administrative users, protecting operational safety.
- Document the hardware communication protocol in a clear, reusable appendix.
- Lay the groundwork for future expansion, including a Modbus TCP test module and an Excel-based bulk data import/export facility.

The successful completion of these objectives yields a centralised, maintainable, and scalable configuration platform that bridges the gap between field hardware and SCADA-level management, ultimately contributing to the reliable operation of passenger information systems in metro rail networks.

Chapter 2

REQUIREMENT SPECIFICATION

2.1 SOFTWARE REQUIREMENTS

- **Operating System** – Windows 10 version 1809 or later (64-bit).
- **Runtime** – .NET 10.0 Desktop Runtime.
- **Database** – SQL Server Local DB 2022 (or SQL Server Express).
- **Development Tools** – Visual Studio 2022, NuGet package manager.
- **Libraries** – Microsoft.Data.SqlClient, System.IO.Ports, System.Net.Http, System.Text.Json, System.Security.Cryptography.
- **Arduino Environment** – Arduino IDE with ESP32 board package, Arduino Json, Preferences, and Webserver libraries for firmware development.
- **Network** – Ping and HTTP connectivity to the target ESP32 devices on port 80.

2.2 FUNTIONAL REQUIREMENTS

Main Window Interface

- MenuStrip control for the top navigation bar
- Panel controls with Dock and Anchor properties for left navigation and central content areas
- Dynamic UserControl loading into the central Panel

Authentication Module

- Modal Login Form with TextBox controls (password masked)
- SHA-256 hashing via System.Security.Cryptography
- AuthRepository class with parameterised SQL queries (Microsoft.Data.SqlClient)

Node Management Module

- ComboBox for route selection
- Button controls for Add, Edit, and Delete actions
- DataGridView for tabular node display
- Modal AddEditNodeForm with TextBox and ComboBox controls
- NodeRepository class for CRUD operations (Microsoft.Data.SqlClient)

Area Management Module

- ComboBox for route selection
- DataGridView populated via LEFT JOIN query (Microsoft.Data.SqlClient)
- Modal EditAreaForm with read-only and editable TextBox controls
- AreaRepository class for description updates

Panel Settings Module

- Synchronised ComboBox controls for panel location and IP address selection (mutual update guard)
- TextBox controls for read-only description and panel serial number
- CheckBox for OTA mode toggle
- SerialPort class (System.IO.Ports) for USB communication
- HttpClient class (System.Net.Http) for OTA communication
- Manual Calibrate Panel (hidden, admin-only) with three TextBox controls
- JSON serialisation via System.Text.Json

Settings Module

- TextBox controls for technician details
- CheckBox and ComboBox for maintenance notification
- JSON file storage via System.Text.Json

Network Configuration Module

- TextBox controls for Ethernet and Wi-Fi parameters
- CheckBox to toggle Wi-Fi fields
- JSON file storage via System.Text.Json

Placeholder Modules (I/O Activity, Upload/Download, Tests)

- Empty UserControl design with label text for future development

Chapter 3

SYSTEM DESIGN

3.1 System Architecture

The User Interface is a Windows Forms application with one main window. The window contains a menu bar at the top, a panel of buttons on the left, and a central area where the current module is shown. Every feature – logging in, managing nodes, describing areas, deploying settings, changing application preferences – is kept in its own separate UserControl. This keeps the main form clean and each module focused on its task. When a user clicks a button in the left panel, the corresponding UserControl is loaded into the central area, replacing the previous one

Behind the user interface, the Application Logic enforces the rules of the system. A set of repository classes (NodeRepository, AreaRepository, AuthRepository) handle all database operations and data validation. They check that IP addresses are unique, that panel serial numbers are not duplicated, and that a panel location is only used once on a given route. The authentication logic hashes passwords with SHA-256 and compares them against stored hashes. The logic also manages the Calibration flag – a node’s status changes from “not calibrated” to “calibrated” only after a successful deployment. The Panel Settings module uses the application logic to synchronize two dropdown lists and to enable the manual calibration panel only for administrators

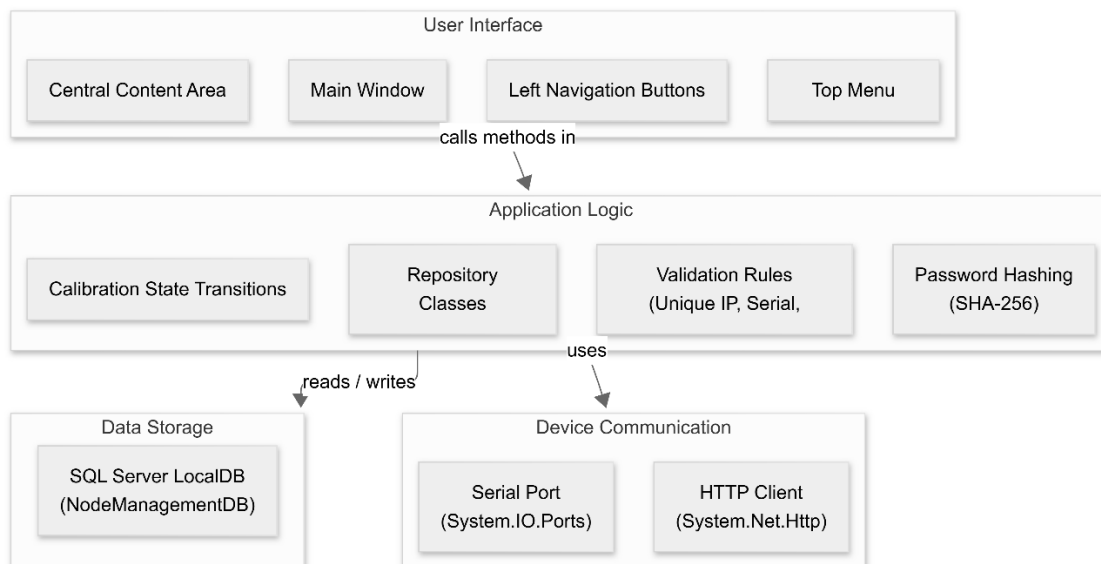


Figure 3.1: System Architecture

The Data Storage part uses SQL Server LocalDB for structured data and a JSON file for simple settings. All database queries use parameterised SQL to prevent injection attacks. A single DatabaseHelper class provides the connection string, making it easy to change the database location later if needed. The JSON file (settings.json) stores technician contact details and maintenance reminders, using the .NET System.Text.Json library for fast, clean serialization.

Finally, the Device Communication part handles talking to external hardware. This is kept entirely inside the Panel Settings module. Two communication methods are available: serial over USB and HTTP over the network. For serial, the .NET System.IO.Ports library opens a COM port, sends a JSON string, and listens for incoming text, which is shown in a live monitor box. For network communication, System.Net.Http.HttpClient sends the same JSON to the device's IP address. Both methods share the same logic for building the JSON payload, and the rest of the application doesn't need to know which method is being used.

3.2 FLOW DIAGRAMS

Application Startup and Login Flow

When the program starts, it shows the main window with almost everything greyed out. The only things a user can click are the Login menu, the Help menu, and the About box. Choosing Login opens a small window asking for a username and password. The program checks these against the database. If they match, the main window comes alive – the buttons on the left side work, the File and Data Transfer menus appear, and the user's name is shown at the top right. If the login fails, an error message appears and the user can try again. Later, clicking Logout will return the window to its locked state after a confirmation.

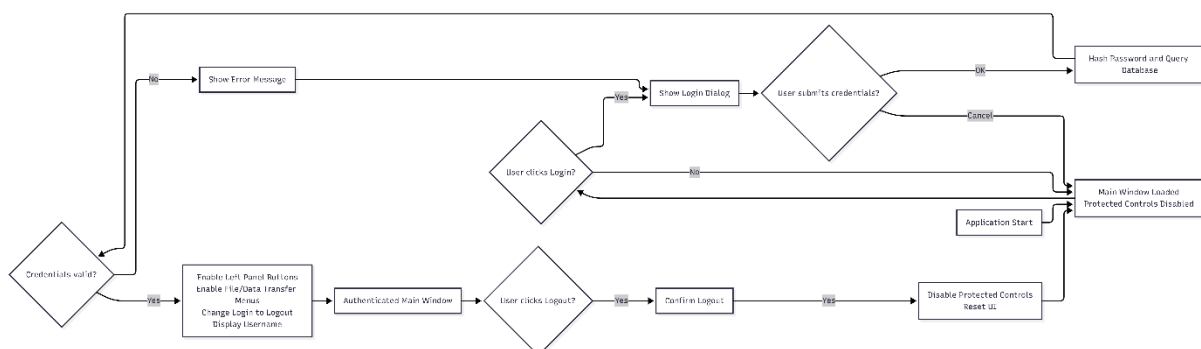


Figure 3.2: Application Startup and Login Flow

Node Configuration and Deployment Flow

This diagram follows the life of a node record from creation to final configuration. It starts in the Node Management module, where a user adds a new node with a unique serial number, IP address, route, panel location, and an optional description. The program checks that all the information is valid and doesn't conflict with existing records. Once saved, the node appears in a grid with a "not calibrated" flag. The user then moves to the Panel Settings module, where

only uncalibrated nodes are shown. After selecting a node, an administrator can optionally enter calibration values (like sensitivity and battery settings). The user then picks how to send the configuration – either by USB serial or over the network – and clicks Deploy. The application builds a JSON message, leaving out any calibration fields that were left empty. If the deployment succeeds, the node is marked as calibrated and disappears from the deployment list, though it still lives in the Node Management grid.

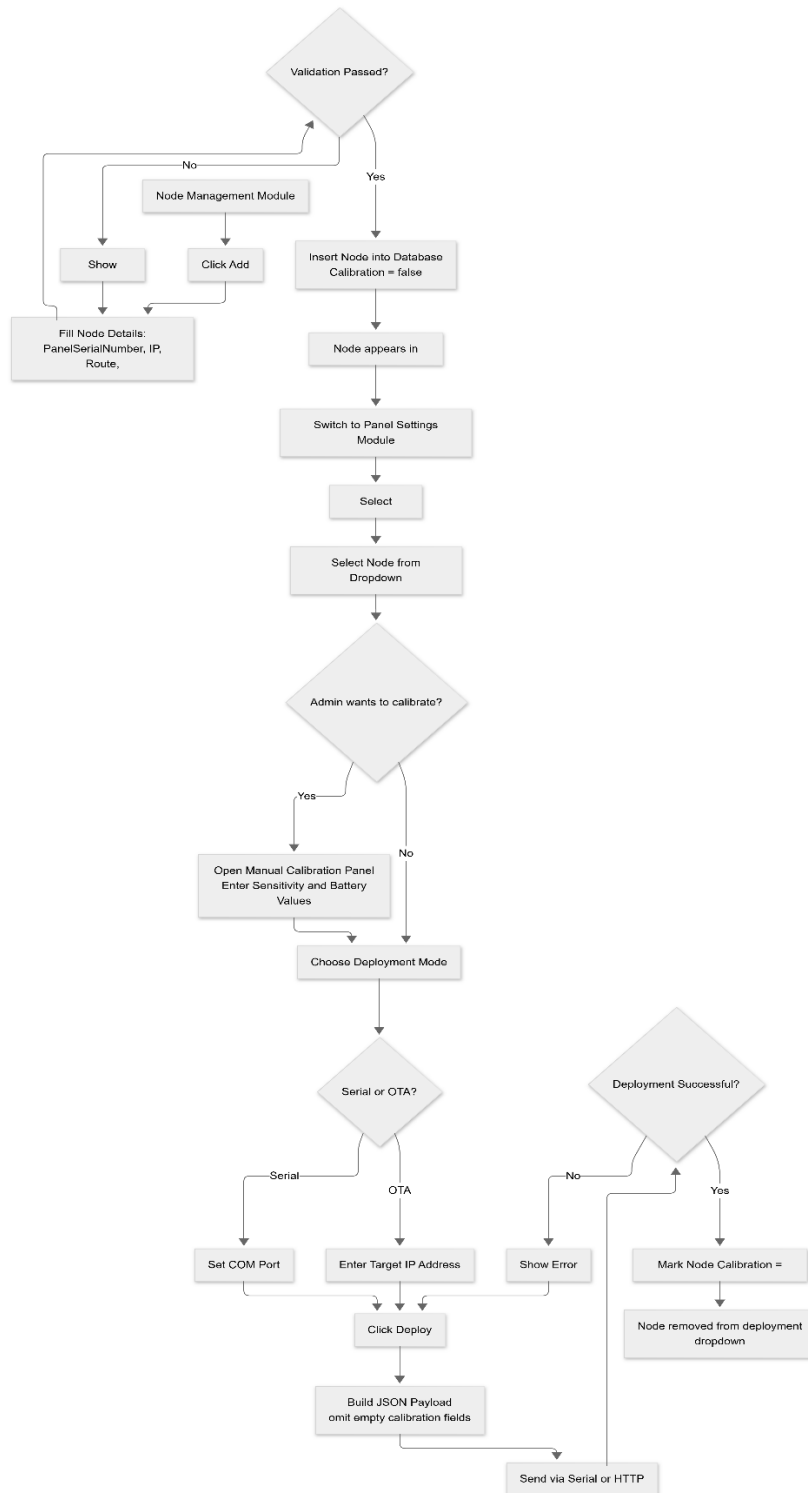


Figure 3.3: Node Configuration and Deployment Flow

Device Communication Sequence

This diagram zooms in on the actual data exchange during a deployment. In serial mode, the application opens a COM port, sends the JSON text, and waits for a reply, which is then shown in the built-in serial monitor. In network (OTA) mode, the application does an HTTP POST to the device's address and waits for a success or error response. In both cases, the device receives the JSON, updates its internal values, saves them permanently, and may restart if the IP address was changed. The same pattern is used for a factory reset, except the JSON contains only a single flag telling the device to clear everything and restart.

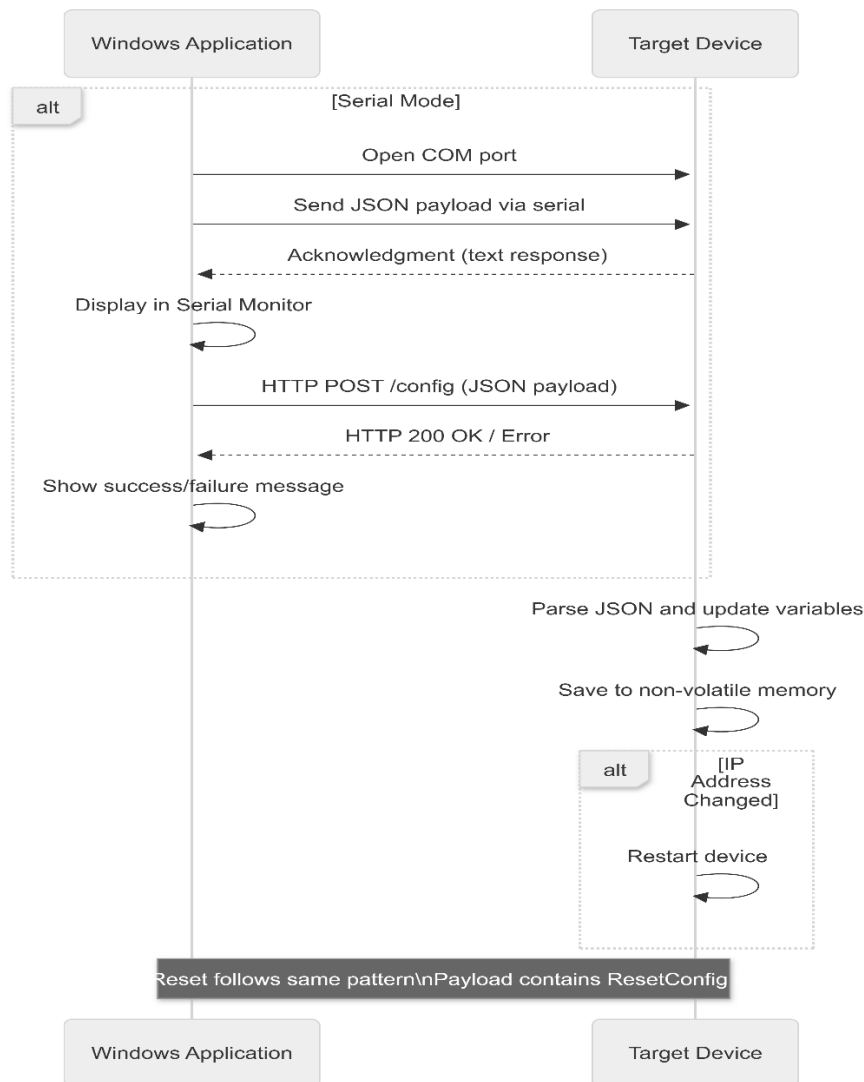


Figure 3.4: Device Communication Sequence

3.3 DATABASE SCHEMA

The application stores its data in a local SQL Server database named NodeManagementDB. Inside this database there are three tables: Users, Areas, and Nodes.

The Users table contains the login information for the people who can open the application. It stores a unique username, a full name that is displayed after login, and a password that has been scrambled using a one-way mathematical function so the original password is never saved. Two accounts are created automatically when the database is first set up: one for an administrator and one for a regular operator.

The Areas table describes every physical panel slot in the system. There are four routes, and each route has one hundred panel locations, making four hundred slots in total. The table is filled with these four hundred rows right from the start. Each row contains a route number, a panel location number, a holding register number that follows the pattern 40001, 40002 and so on, and a unique panel serial number that matches the number engraved on the hardware. A description field is also available, initially empty, where a technician can write the actual location of the panel, such as “Platform 3, Column B”. The serial number and the holding register are guaranteed to be unique across the whole table, and the combination of route and panel location is also guaranteed to be unique.

The Nodes table holds all the detailed settings for a device that has been installed at a particular panel slot. Each row in Nodes is linked to a row in Areas by the panel serial number. Besides the serial number, the table stores a user-friendly node number, the IP address assigned to the device, and many calibration values. These calibration values include the zero-voltage offsets for two current sensors, the sensitivity of each sensor, the current thresholds that decide whether a sensor reading is good or bad, the battery calibration factor, a compensation value for battery voltage drops, and the power supply threshold. All these calibration numbers start with sensible default values, so a node can be added without entering everything straight away.

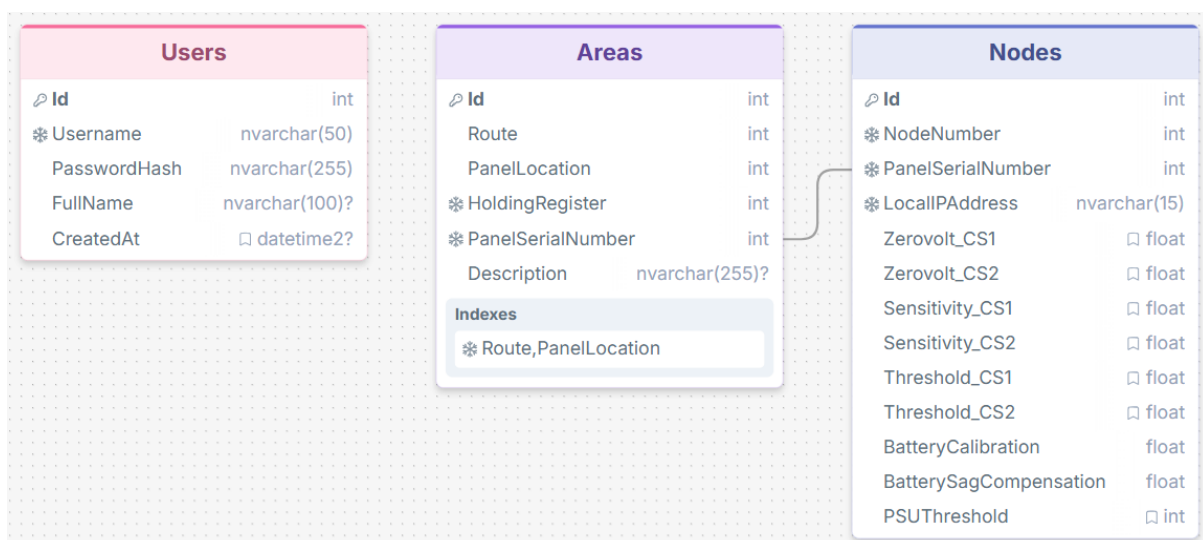


Figure 3.5: Database Schema

Chapter 4

SYSTEM IMPLEMENTATION

4.1 MAIN WINDOW INTERFACE

The main application window uses a familiar three-area layout. At the top sits a menu strip, on the left a vertical stack of buttons, and the centre is a large panel where the active module appears. When the program starts, the central area shows the company logo. All navigation buttons are greyed out until the user logs in, keeping the protected features inaccessible to unauthorised persons. The window can be resized, and the layout adapts because the panels use docking and anchoring properties.

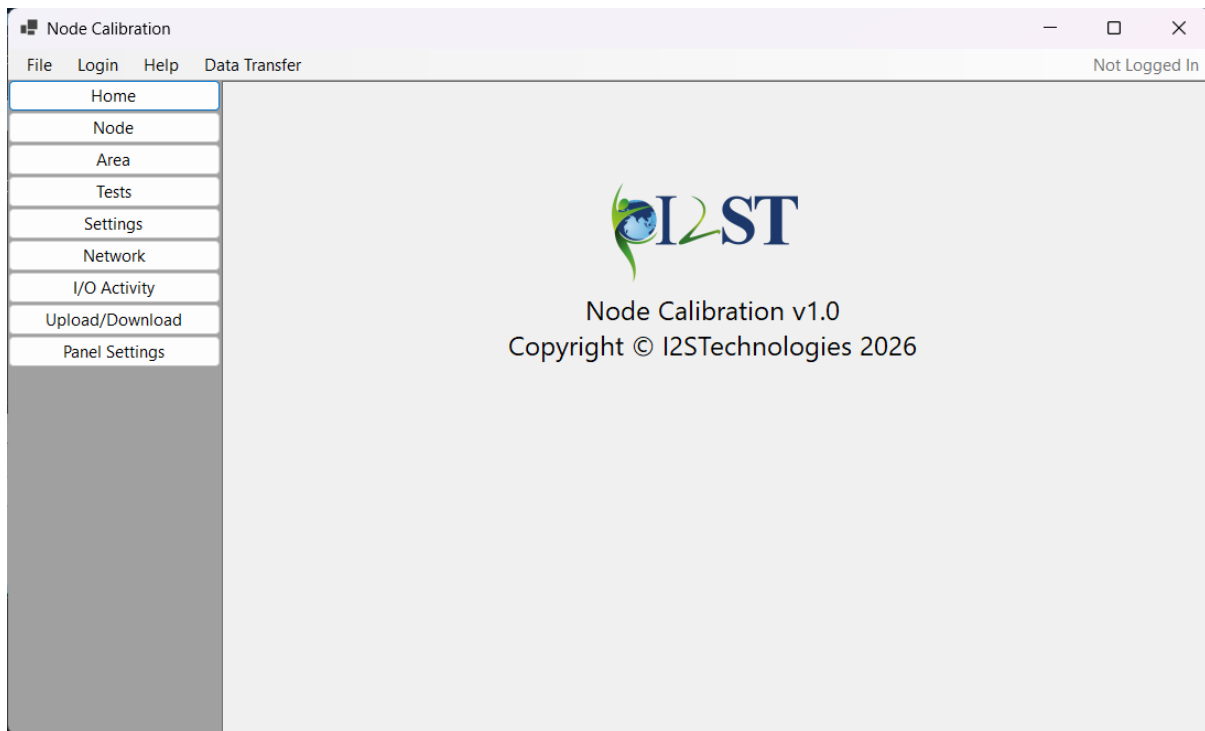


Figure 4.1: Main Windows Form.

4.1.1 Menu Strip (Top Navigation Bar)

The menu strip at the top provides quick access to global commands. It contains four main menus: **File**, **Login**, **Help**, and **Data Transfer**. The File menu lets the user create a new specification file, open an existing PDF, save, or print. The Login menu changes its text depending on the login state – it reads “Login” when nobody is signed in, and “Logout” when a user is already authenticated. The Help menu offers a User Manual that opens a PDF file and an About box that shows the program’s version. The Data Transfer menu is reserved for future

upload and download features. At the far right of the menu strip, a small icon and a label display the current user's name once logged in, otherwise it shows "Not Logged In".

4.1.2 Left Navigation Panel

The left panel is a vertical strip of buttons, each labelled after a main function. The buttons are: Home, Node, Area, Tests, Settings, Network, I/O Activity, Upload/Download, and Panel Settings. Clicking a button changes the centre panel to the matching module. The Home button returns the central area to the company logo. All buttons are disabled until a successful login, ensuring that nobody can access the modules without permission. This panel remains visible at all times, giving constant quick access to every part of the program.

4.2 AUTHENTICATION MODULE

The login dialog appears when the user clicks the Login menu item. It is a small, fixed-size window with two text fields (username and password), a Login button, and a Cancel button. The password is hidden as asterisks while typing. When the user presses Login, the program scrambles the password with a one-way hash and compares it against the database. If the credentials match, the dialog closes, the main window becomes fully active, the left panel buttons light up, the Login menu changes to Logout, and the user's full name appears at the top right. If the login fails, an error message is shown and the dialog stays open. Two accounts exist by default: an administrator ("admin") and a standard operator ("Rakesh S"). The administrator can access extra features, such as calibration controls, that remain hidden from other users.

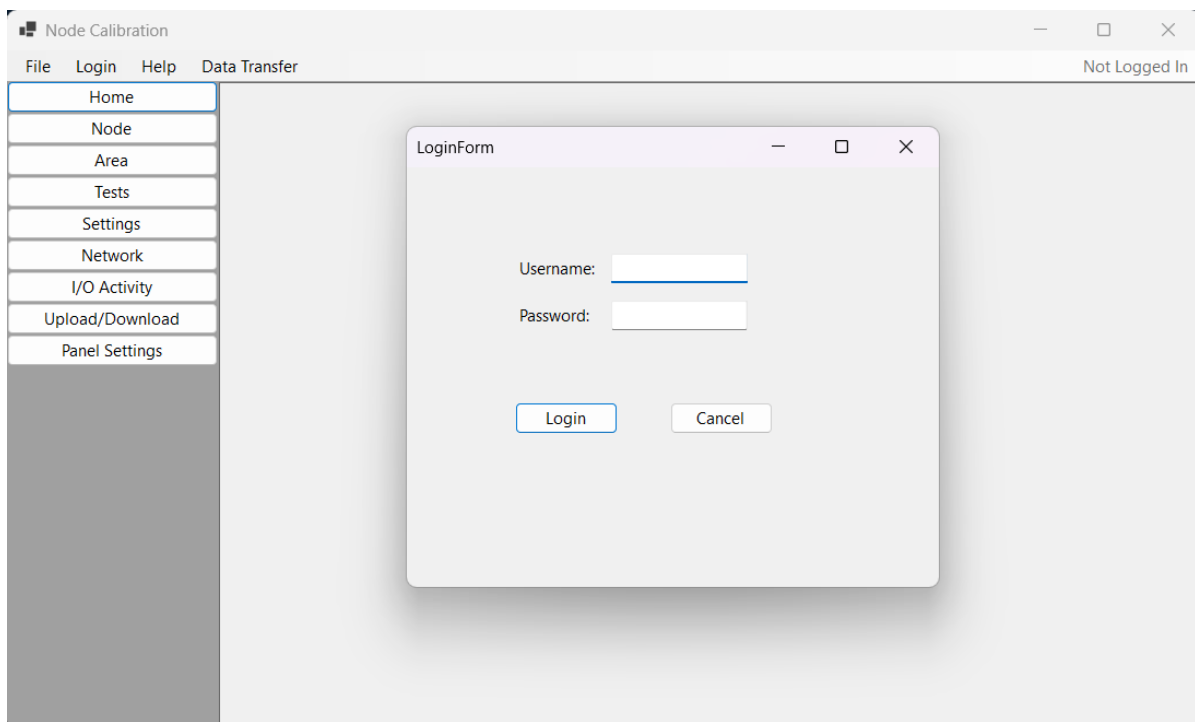
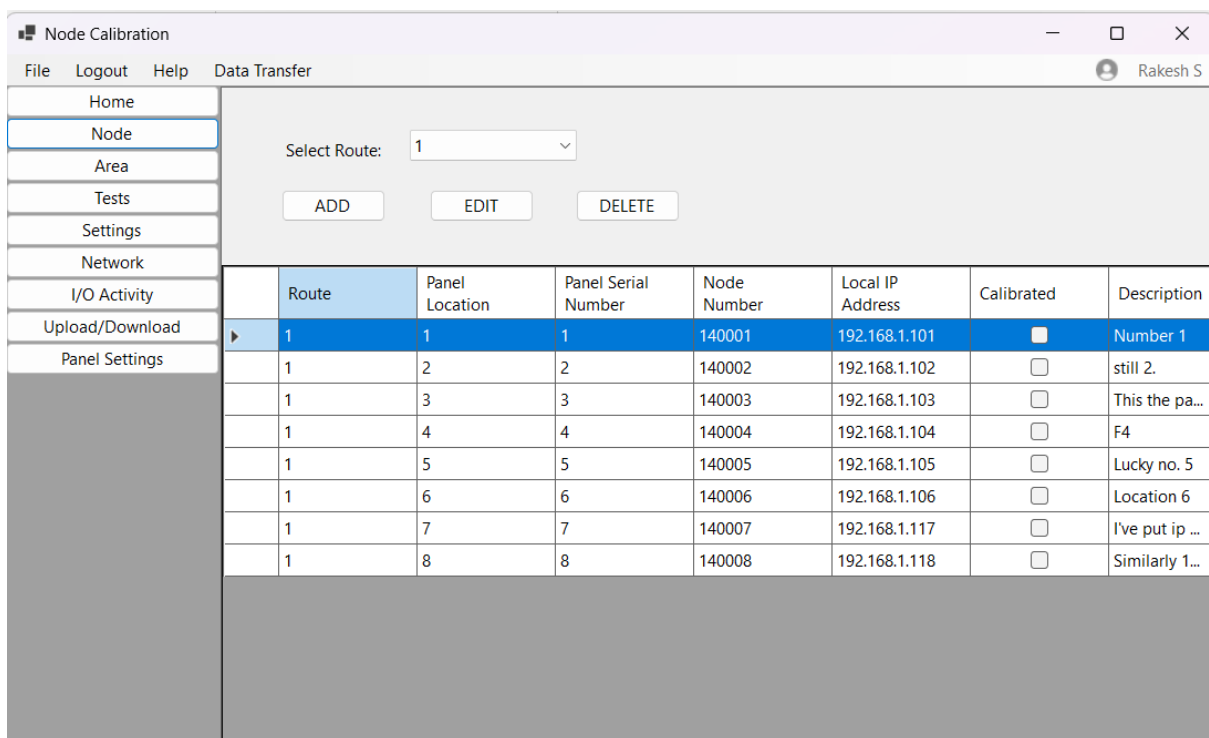


Figure 4.2: Login Form.

4.3 NODE MANAGEMENT MODULE

The Node module is the place for managing device records. It contains a dropdown at the top for selecting a route (1 to 4), three buttons (Add, Edit, Delete), and a data grid that fills the rest of the area. The grid displays every node belonging to the selected route, showing the route number, IP address, panel location, description, a checkbox for calibration status, and the panel serial number. Clicking **Add** opens a new dialog where the user must enter a unique panel serial number, an IP address, and a panel location number, plus an optional description and calibration flag. The program checks that the IP address is valid, that it is not already used, that the panel location is free on the same route, and that the serial number is unique. If any check fails, an error message appears and the dialog remains open. **Edit** works the same way but pre-fills the fields with the selected node's current data. **Delete** asks for a confirmation before permanently removing the selected record. All data operations use safe, parameterised database queries.



Route	Panel Location	Panel Serial Number	Node Number	Local IP Address	Calibrated	Description
1	1	1	140001	192.168.1.101	☒	Number 1
1	2	2	140002	192.168.1.102	☐	still 2.
1	3	3	140003	192.168.1.103	☐	This the pa...
1	4	4	140004	192.168.1.104	☐	F4
1	5	5	140005	192.168.1.105	☐	Lucky no. 5
1	6	6	140006	192.168.1.106	☐	Location 6
1	7	7	140007	192.168.1.117	☐	I've put ip ...
1	8	8	140008	192.168.1.118	☐	Similarly 1...

Figure 4.3: Node User Control.

4.4 AREA MANAGEMENT MODULE

The Area module helps technicians label the physical location of every panel slot. It displays a route dropdown and a data grid. When a route is picked, the grid shows all one hundred panel locations for that route, each with its panel number, the IP address of any node assigned there (or “Not Assigned” if vacant), and a description field. The description is initially empty. To add or change a description, the user selects a row and clicks the **Edit** button. A small dialog opens showing the panel location number (which cannot be changed) and a text box for the

description. Saving updates the database. The grid itself cannot be directly edited; all changes go through the Edit dialog. The data is retrieved by combining the Areas table (which holds the descriptions) with the Nodes table (which holds the IP addresses) through a left-join query.

Route	Panel Location	Holding Register	Panel Serial Number	Panel Location Description
1	1	40001	1	Number 1
1	2	40002	2	still 2.
1	3	40003	3	This the panel 3 loc, it next to KR bridge
1	4	40004	4	F4
1	5	40005	5	Lucky no. 5
1	6	40006	6	Location 6
1	7	40007	7	I've put ip 117 for this location 7
1	8	40008	8	Similarly 118
1	9	40009	9	
1	10	40010	10	
1	11	40011	11	
1	12	40012	12	
1	13	40013	13	

Figure 4.4: Area User Control.

4.5 PANEL SETTINGS MODULE

This module is used to send configuration to a hardware device. It only shows nodes that have not yet been calibrated. The top part has a route selector, a panel location dropdown, and an IP address dropdown that stay linked together. Two read-only fields show the panel description and the panel serial number. There is a checkbox labelled “OTA Deployment” that switches between two modes: USB serial and over-the-air via HTTP. In serial mode, the user sees a COM port selector, Connect/Disconnect buttons, and a large serial monitor area that shows live text from the device. The Deploy button is disabled while the serial port is open; the user must disconnect before deploying. In OTA mode, the serial controls are replaced with a text field for the device’s IP address, and Deploy sends an HTTP request instead. A **Manual Calibrate** button, visible only to the administrator, opens a hidden panel with three calibration fields: ACS Sensitivity 1, ACS Sensitivity 2, and Battery Calibration. If these fields are filled, the values are included in the JSON payload; if left empty, they are omitted. After a successful deploy, the node is marked as calibrated and disappears from the dropdown. A **Reset Config** button sends a special JSON command that wipes all stored settings on the device and restarts it to factory defaults.

The screenshot shows the 'Node Calibration' application window. The sidebar menu on the left includes 'Home', 'Node', 'Area', 'Tests', 'Settings', 'Network', 'I/O Activity', 'Upload/Download', and 'Panel Settings'. The main content area is titled 'Panel Settings' and contains the following elements:

- Route:** A dropdown menu with '1' selected.
- IP Address:** A text box containing '192.168.1.101'.
- Description:** A text box containing 'Number 1'.
- Panel Location:** A dropdown menu with '1' selected.
- PSN:** A text box containing '1'.
- Ethernet:** A checkbox that is currently unchecked.
- Esp32 IP Address:** An empty text box.
- Deploy:** A button.
- Serial Monitor:** A section with 'Connect' and 'Disconnect' buttons, and a 'Serial Port' dropdown menu.

Figure 4.5: Panel Settings User Control.

4.6 SETTINGS MODULE

The Settings module stores general application preferences. It shows four text boxes for the technician's name, phone number, email, and address. Below these, a checkbox enables maintenance notifications. When the checkbox is ticked, a dropdown becomes available that lets the user pick a reminder frequency: one month, three months, six months, or twelve months. All values are saved to a JSON file on the computer, and they are loaded again each time the module is opened. This module does not use the database; it works entirely with the local file.

The screenshot shows the 'Node Calibration' application window with the 'Settings' module selected in the sidebar. The main content area contains the following elements:

- Technician Name:** A text box.
- Phone Number:** A text box.
- Email:** A text box.
- Address:** A text box.
- Enable Maintenance Notification:** A checkbox that is currently unchecked.
- Reminder Frequency:** A dropdown menu.

Figure 4.6: Settings User Control.

4.7 NETWORK CONFIGURATION MODULE

The Network module displays and allows editing of the host computer's network settings. It shows Ethernet parameters (IP address, subnet mask, and default gateway) as three labelled text boxes. A checkbox labelled "Wi-Fi Enabled" opens an extra panel with fields for the Wi-Fi SSID, password (masked), IP address, subnet mask, and default gateway. All these values are saved to the same JSON file used by the Settings module. The information is for reference and documentation; it does not automatically change the computer's real network configuration.

The screenshot shows a web application window titled "Node Calibration". The interface has a top navigation bar with "File", "Logout", "Help", and "Data Transfer" links, and a user profile "Rakesh S". A left sidebar contains a menu with items: Home, Node, Area, Tests, Settings, Network, I/O Activity, Upload/Download, and Panel Settings. The main content area is titled "Network Settings" and contains two sections. The first section, "Ethernet Settings:", includes three text input fields for "IP Address:", "Subnet Mask:", and "Default Gateway:". Below these is a checkbox labeled "Wifi Enabled" which is checked. The second section, which is expanded, includes five text input fields for "SSID:", "Password:", "IP Address:", "Subnet Mask:", and "Default Gateway:".

Figure 4.7: Network Settings User Control.

4.8 I/O ACTIVITY MODULE

This module is a placeholder for future development. Its intended purpose is to display live input/output data, such as sensor readings and relay states, from connected devices. In the current version of the application, it loads a blank page with its name displayed, ready for later implementation when the monitoring feature is added.

4.9 UPLOAD/DOWNLOAD MODULE

The Upload/Download module is also a placeholder. It will eventually allow users to import node and area data from an Excel spreadsheet, which is useful when setting up a large number of devices at once. It will also provide an export function to download the current database

contents into an Excel template for offline editing. For now, the module shows only a label indicating that the feature is under development.

4.10 TESTS MODULE

The Tests module is planned for performing diagnostic checks on devices using the Modbus TCP protocol. It will let an operator select one or more nodes and run test sequences, such as toggling relays or reading sensor values, and see the results in real time. Like the I/O Activity and Upload/Download modules, it currently appears as a placeholder with its name, waiting for future coding.